Written Assignment for Course:

DLMDSPWP01 – Programming with Python

Paul Hinrichs

IU14140227

Submitted on 24.02.2026

Table of Contents

List of figures III

1 Introduction 1

1.1 Scope of the program 1

1.2 Overview of the document structure 1

2 Assignment and Requirements 2

2.1 Original assignment description 2

2.2 Functional requirements 2

2.3 Non-functional requirements 3

2.4 Data Requirements 3

2.5 Technical Specifications 4

3 System Overview 5

3.1 High-level architecture 5

3.2 Code Structure 6

3.3 Main technologies and libraries used 6

3.4 Data flow in the program 6

4 Design and Implementation by Requirement 7

4.1 Implementation of functional requirements 7

4.1.1 FR1: Data import from CSV 7

4.1.2 FR2: Comparison of mathematical functions 7

4.1.3 FR3: Comparison of mathematical points 8

4.1.4 FR4: Data visualization 9

4.1.5 FR5: Compile SQLite database 11

4.2 Implementation of non-functional requirements 12

5 User Perspective 14

5.1 Installation and setup 14

5.2 Usage guide 15

6 Testing and Validation 16

6.1 Test strategy 16

6.2 Test cases linked to requirements 17

6.3 Known limitations and bugs 18
7 Conclusion and Future Work 19
7.1 Summary of what was achieved 19
7.2 Lessons learned 19
7.3 Possible extensions or improvements 20
7.4 Possible use cases 20
Appendix 21

List of figures

Figure 1 UML-diagram of the high-level class structure 5

1 Introduction

This document explains the design, implementation and usage of “dataset_function_comperator”, a Python application developed for the written assignment of the “Programming with Python” course. The program fulfills the requirement to process and visualize the mathematical data from a CSV-file.

1.1 Scope of the program

The program reads three CSV-files provided with the assignment. These files contain datasets representing two-dimensional mathematical functions and points. The first dataset contains ideal functions, the second training functions and the third test points. Each dataset is transformed and encapsulated into a separate readable table object. Afterwards comparisons of the functions and points are computed with the table objects. All data is displayed as plots in a web browser to allow user interactive visualization. In this way, the user can visually analyze the data and draw conclusions from the comparisons. Finally, the resulting mappings and all intermediate tables are stored in an SQLite database to enable later inspection or reuse in other analysis tools.

1.2 Overview of the document structure

Chapter 2 first presents the assignment, describes the functional requirements derived from it, addresses the non-functional requirements implied by the assignment, and finally explains the data- and technical-specifications. Chapter 3 provides an overview of the system that implements these requirements. This includes a visualization of the object-oriented structure of the program with a UML-diagram, a description of the libraries used, and an explanation of the main processing steps. Chapter 4 describes how each requirement is implemented in the program code. Each requirement subsection combines a brief restatement of the requirement with a detailed explanation of the implementation. Chapter 5 explains how the program is intended to be set up and used. Chapter 6 presents the test strategy and the known limitations of the program, and links test cases explicitly to the functional requirements they verify. Finally, Chapter 7 summarizes what was achieved with the implementation, what was learned in the process, what could be improved, and what further use cases could be.

2 Assignment and Requirements

This chapter describes the underlying course assignment in section one and then explains the derived functional, non-functional and data-related requirements as well as technical specifications in the following sections.

2.1 Original assignment description

Three datasets in form of CSV-files are given for the assignment. A program must be implemented that can read the files, do two calculations based on the content of the datasets and then visualize the results.

The first calculation is a comparison of the mathematical functions found in two of the three datasets. The task is to find the four functions in the first dataset containing 50 functions that are closest to the four functions in the second dataset.

The second calculation is a comparison of points on a mathematical graph with the four mathematical functions found. The third dataset contains points, which must be in a certain range from one of the four functions. If they are, the function and the distance to the function must be saved with the third dataset.

2.2 Functional requirements

FR1: Data import from CSV

The system must read CSV-files and save the content. It also must apply the logical structure of the CSV-files to the representation of the content for later access.

FR2: Comparison of mathematical functions

The system must compare four two-dimensional mathematical functions with 50 other two-dimensional mathematical functions based on the y-deviations squared between them. Result of the comparison must be the identification of the functions which have the minimal y-deviations squared. The identified functions must be saved, as they are needed for later access.

FR3: Comparison of mathematical points

The system must compare a table of two-dimensional mathematical points with a table of two-dimensional mathematical functions (All mathematical functions that are to be tested must have a maximum y-deviation saved from a comparison of functions). Every point must be compared with every function. If the regression between a point and a function is smaller than the square root of the maximum y-deviation for that function, the function and the regression must be saved with the point in its table.

FR4: Data visualization

All data must be visualized. This includes the mathematical functions in the first two datasets and the mathematical points in the third dataset, as well as all computed results.

FR5: Compile SQLite database

The three tables from each CSV-file containing the mathematical information must be written into a SQLite database. The first dataset with 50 mathematical functions must be compiled in a single 51 column spreadsheet, the second dataset with four mathematical functions must be compiled in a single five column spreadsheet. In both cases the first column represents the x-values of the two-dimensional functions, and all following columns represent the y-values of the two-dimensional functions. The third dataset with the two-dimensional points must first be compared with the mathematical functions as described in FR3. Then the resulting table must be compiled in a four-column spreadsheet, with column one holding the x-values, column two holding the y-values, column three holding the distance between point mapped function and finally column four holding the function the point was mapped to.

2.3 Non-functional requirements

NFR1: Object-oriented design

The system design must be object-oriented and use appropriate modularization.

NFR2: Inheritance

The system shall include at least one inheritance between classes.

NFR3: Exception handling

The system must handle standard and user-defined exceptions.

2.4 Data Requirements

The three datasets for the program were given with the assignment, and the program is designed to work with these datasets. The program must be object-oriented as NFR1 specifies and therefore can handle new datasets, if their structure is equivalent to the structure of the given datasets. The given datasets have the following structure:

DS1: Ideal functions

The CSV-file contains one line functioning as a header with function-names and 400 lines of float-values. Every name and value is separated by a comma. The first value in each line stands for the x-value of all mathematical-functions in the file. Following the x-values are exactly 50 y-values for 50 mathematical functions. The x-values reach from -20.0 till 19.9 in 0.1 intervals.

DS2: Train functions

The CSV-file contains the identical structure as the DS1, with the difference that DS2 only contains four functions instead of 50.

DS3: Test Points

The CSV-file contains one line as a header and 100 lines of two float-values separated by a comma. Each line holds an x-value followed by a y-value, representing two-dimensional points in a mathematical graph. The points have no order.

2.5 Technical Specifications

The following technologies are mandatory for the implementation of the assignment:

- Python must be used as the programming language.
- Pandas library must be used for data manipulation and DataFrame operations.
- SQLAlchemy library should be used for saving data in a database.
- SQLite should be used for the local database engine.
- Bokeh library must be used for interactive logical visualization of data.

3 System Overview

Chapter three explains the structure of the system implementing the assignment. The first section describes how the different classes of the system interact with each other. What code is written in what file is presented in section two. Section three explains what technologies were used in the implementation. The last section shows how the data is processed through the system, from input to output.

3.1 High-level architecture

The architecture consists of a shared parent class for table data, two specialized subclasses for mathematical functions and points, and a visualizer component that displays their contents. The following UML-diagram shows the architecture of the system on a high level.

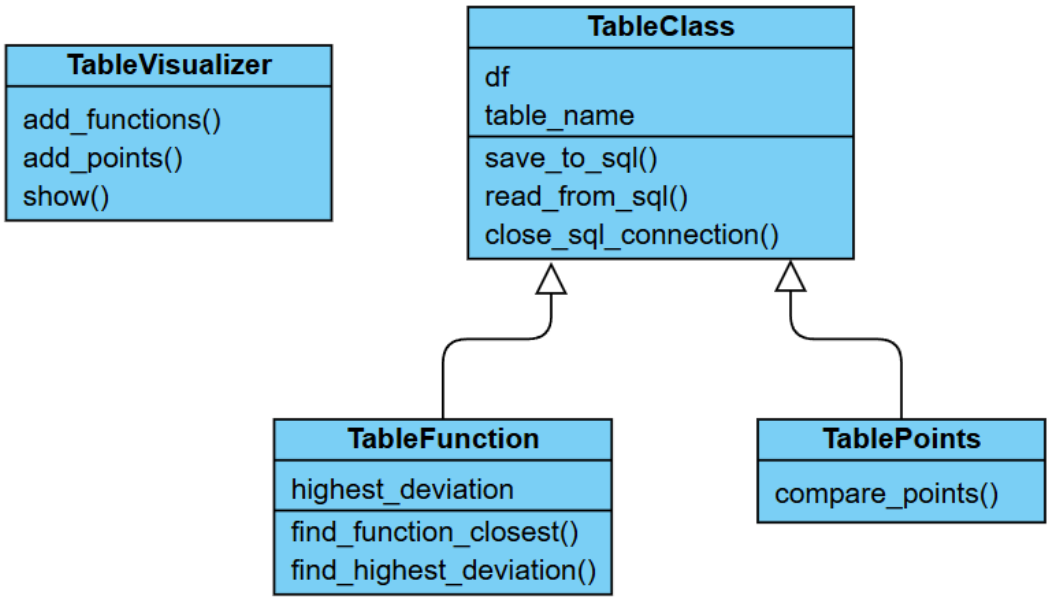

Figure 1 UML-diagram of the high-level class structure

The main class is TableClass, which has two attributes. The attribute df holds the data from the dataset for which the class instance is created, and table_name stores the name for the table which is used in the database.

TableFunction inherits from TableClass and is intended for tables that contain data representing mathematical functions. It adds the attribute highest_deviation. The deviation is only calculated for functions and is therefore only needed in this subclass. The method find_function_closest() is the main method for TableFunction and takes a second TableFunction instance as a parameter.

TablePoints also inherits from TableClass. It is used for tables that contain data representing two-dimensional mathematical points. The method compare_points() is added in this subclass and takes a TableFunction instance as parameter.

TableVisualizer creates a single visualization for each class instance, which can be displayed with the method show(). Before the visualization is shown, data must be added using either add_functions(), which takes a TableFunction instance as parameter or add_points(), which takes a TablePoints instance as parameter.

3.2 Code Structure

The project follows a src/test layout: all application code lives in the src package, and automated tests with their test data live in a separate top-level test directory.

The src directory is a Python package and holds the code implementation:

- main.py contains the entry point and orchestrates loading data, creating objects, and creating visualizations.
- tables.py contains TableClass and its subclasses TableFunction and TablePoints.
- visualizer.py contains TableVisualizer responsible for plotting tables and points.
- src/data/ contains the input CSV files that are loaded by the table classes.
- src/visualization/ is the output folder where generated HTML plots are stored.

The test directory contains unit tests and their dedicated test data:

- test_tables.py groups all unit tests for TableClass, TableFunction and TablePoints.
- test/data/ stores small CSV fixtures used only for testing.

3.3 Main technologies and libraries used

The project uses Python 3.14 with the pandas library for data manipulation, SQLAlchemy for database access, and Bokeh for interactive data visualization, in accordance with the assignment specifications (see Section 2.3).

3.4 Data flow in the program

1. The program creates wrapper-class instances of TableFunction and TablePoints for the dataset. These instances read the input data and load them into pandas DataFrame objects. They provide uniform interfaces for mathematical functions and points.
2. The tables for mathematical functions are compared and analyzed to determine deviations between them. The result is wrapped in a new instance of TableFunction.
3. The table for mathematical points is then compared with the result table containing the compared mathematical functions.
4. The results of both comparisons, as well as the initial tables are each passed to instances of TableVisualizer.
5. The visualizations are rendered and displayed for the user.
6. Finally, all data is saved into SQLite datasheets.

4 Design and Implementation by Requirement

This chapter explains in detail how the requirements and the design were implemented. First the implementation of functional requirements gets described, then the implementation of non-functional requirements.

4.1 Implementation of functional requirements

Section 4.1 explains each implementation of the functional requirements from section 2.2.

4.1.1 FR1: Data import from CSV

The format of data in the CSV files (see section 2.4) can be read and directly converted into a pandas DataFrame using the pandas method read_csv(). The program makes use of this, as the use of pandas is required for the task.

The class TableClass and its subclasses act as wrapper classes to allow efficient and object-oriented work with the underlying DataFrame. The CSV file is read in the constructor method of TableClass; therefore, the relative path of the file to be read must be provided as a parameter.

In main.py, an instance of TableFunction or TablePoints is created for each file. The constructor is called with fixed CSV file paths, since the program is not required to work with other CSV files. The object-oriented structure of the program would allow the use of different CSV files, but user input for file selection would have to be implemented, which is not part of the requirements.

4.1.2 FR2: Comparison of mathematical functions

The requirement specifies that four two-dimensional mathematical functions must be compared with 50 other functions. Since the comparison is limited to functions, the corresponding method is implemented in the subclass TableFunction. It is named find_function_closest() with two parameters:

1. An integer function_index that represents the position of the function in the table for which the method is called. This singular function is compared to multiple other functions.
2. A TableFunction instance that must contain at least one function and can theoretically contain many functions. The required computing power is scaling with the size of this TableFunction.

In Main.py, a for-loop iterates over all functions in the TableFunction instance that is initialized for the training dataset. Inside the loop, find_function_closest() is called on this instance of TableFunction. The first parameter uses the loop variable, so that the method is called for every function in the table. The second parameter is the TableFunction that holds the data for the 50 ideal functions. A dictionary is initialized before the loop and is filled with the result of find_function_closest() in each iteration. This sequence of code thus compares every function in the training dataset with every function in the ideal dataset.

Afterwards, a new TableFunction is initialized with the dictionary that holds the result data and is stored for the next requirement FR3. The construction of a TableClass with a dictionary as data source has been implemented as an alternative to the CSV-file for this case.

The following describes the implementation of the method find_function_closest():

A loop iterates over all columns of the table passed as the parameter, where each column represents a mathematical function. The first column is skipped because it contains the x-values shared by all functions. For each remaining column, the deviation between the function to be compared and the current function in the loop is calculated.

The calculation of the deviation between two functions is implemented in the method compare_functions() for better readability and reusability of the code. This method iterates over all y-values of both functions in a loop. In each iteration, it computes the difference between the two values, squares it, and adds it to a running total that is initialized before the loop. At the end, the method returns this total sum.

Inside find_function_closest(), this return value is stored for the current iteration and then compared with the lowest deviation sum found in previous iterations. If the current deviation sum is lower, it is saved as the new minimum. In this case, the y-values of the corresponding function and its name are also stored as the current best match.

After all iterations have completed, the method returns a list containing the name of the function with the lowest deviation sum as well as the y-values of this function.

4.1.3 FR3: Comparison of mathematical points

Requirement FR3 specifies that a table containing two-dimensional mathematical points must be compared with a table containing two-dimensional mathematical functions. Specifically, the points in the test dataset must be compared with the four ideal functions identified for FR2. To clearly separate the two types of tables the program works with, TablePoints is implemented as a subclass of TableClass. The method compare_points() is defined in this subclass and takes one parameter, a TableFunction instance. The points contained in TablePoints are compared with all functions contained in this TableFunction.

The comparison of mathematical points with mathematical functions requires that each function has previously been compared to another function, and that the highest deviation of this previous comparison is stored for use in the point comparison. To achieve this, FR2 is executed in main.py first, and then the highest deviation for each of the four resulting functions is calculated using find_highest_deviation().

The method find_highest_deviation() is implemented for TableFunction. It takes two parameters:

1. An integer function_index that represents the position of the function in the table for which the method is called. This singular function is compared with a second function.
2. A pandas Series that contains the second function to be compared. This Series represents a column of a pandas DataFrame.

Inside the method, a loop iterates over both functions and computes the absolute distance between them at each point. In each iteration, this distance is compared with the previously stored maximum distance and, if it is larger, it replaces the current maximum. After all iterations, the maximum distance found is stored as the maximum deviation for the function in the highest_deviation attribute of the TableFunction. The maximum distance is also returned as result of the method for further storing.

After the highest deviation has been calculated for all four functions in main.py, the method compare_points() is called for the TablePoints instance. The method compare_points() has no return value; instead, the result of the comparison is written directly in the TablePoints DataFrame. For this reason, no further processing is required in main.py.

A nested iterating loop is implemented in compare_points(). The outer loop iterates over all points in TablePoints and stores their coordinates for access inside the inner loop. The row in the TableFunction that has the same x-value as the current point is also stored. The inner loop then iterates over the four y-values in this row. For each of those values, the absolute distance between the point and the corresponding value of the current function is calculated. The result is compared with the square root of the maximum deviation that was previously found for that function. If the absolute distance is smaller, the distance and the name of the function are stored in dictionaries.

After all iterations have completed, the two dictionaries that store the mappings from points to functions are concatenated to the TablePoints as new columns.

4.1.4 FR4: Data visualization

The requirement specifies that all data must be visualized in a logically structured way. Implementing this means that three plots must be created for the three datasets, as displaying all data in a single plot would overwhelm the user and would not be practical. In addition, one plot is created for the comparison of functions and further plots are created for the comparison of points with functions.

Dataflow in main.py:

To implement these requirements, a class was created that wraps a figure object from the Bokeh library. The class is called TableVisualization. The methods add_functions() and add_points() are implemented in this class, to add data to the Bokeh figure. These methods are explained in detail, after their usage is described.

One instance of TableVisualization is initialized in main.py for each dataset. Afterwards, the methods add_functions() and add_points() are used to add the data from the datasets, which are wrapped in instances of TableFunction and TablePoints. Finally, for each instance the method show() is called, which displays the figure and all added data in the browser. This implementation achieves minimal code duplication and effective object orientation, without extensive modularizing.

When visualizing the comparison between functions, the second of the four functions is not displayed. The reason is that the y-values of function two range from -16000 to 15700, while the y-values of the other three functions range from -900 to 100. The large scale of function two makes it impossible to display it with the other functions in a meaningful way. Function two is also not shown in a separate plot, because the deviation between the ideal function and the training function is so small, that it is not visible at that scale and the visualization would therefore be pointless.

The visualization of the comparison between points and the four ideal functions has the same scale problem. However, in this case the goal of the visualization is not to show the deviation between functions, but to show that all points map onto the functions. Therefore, the second function, together with all points mapped to it, is shown in an additional plot on its own.

In total, six visualizations are displayed: three for the three datasets, one for the comparison of functions, and two for the comparison of points with functions.

Methods for class TableVisualization:

The constructor method for TableVisualization takes a string parameter that is used as the file name for the HTML-file created when displaying the visualization. A second string parameter is taken used for the title shown to the user to communicate what the plot represents. The constructor additionally creates a Bokeh figure and a tuple of strings. This tuple is filled with seven color names, which are iterated each time data is added to the figure. In this way, the different data is more distinguishable and readability for the user is improved.

The method add_functions() takes two parameters:

1. An instance of TableFunction that contains the functions to be added to the visualization.
2. A set used for the two comparisons visualizations, as an optional parameter. This set must contain integer values representing the indices of the functions in the table that should be displayed. If all functions are to be displayed, the set is omitted when calling the method.

A loop iterates over all columns in the TableFunction except for the first column, which contains the x-values. If the set of indices is provided, each iteration checks whether the current column index is contained in the set. In this case, and also if the set is not provided at all, the first column of TableFunction (containing the x-values) and the current column (containing the y-values) are used to call the implemented method add_function().

This method add_function() uses the line() function from the Bokeh library to add the function via the list of x-values and the list of y-values. This step is extracted into a separate method because Bokeh's line() function requires the given lists to have the same type, the same length and a consistent content type. For readability and reusability, the verification of these requirements is moved to add_function() and is invoked by add_functions().

The method add_points() takes two parameters, analogous to add_functions():

1. An instance of TablePoints that contains the points to be added to the visualization.
2. A set used for the two comparison visualizations, as an optional parameter. This set must contain string values representing the names of functions; only the points in the table that have been mapped to these functions are displayed by this method. If all points in the table should be displayed, the set is omitted when calling the method.

A loop iterates over all points in TablePoints. If the set is provided, each iteration checks whether point is mapped to one of the functions in the set. If this is the case, or if the set is not provided, the current point is added to the figure. The glyph circle with size five is always used, as this was found to be a suitable shape and scale to keep the data readable for the user.

Finally, the method show() takes no parameters. It configures output_file() to use a separate HTML file. This file is stored in the subfile src/visualizations/ This prevents different visualizations from overwriting each other when multiple plots are displayed in quick succession. By creating a separate HTML file for each call, the visualization data is preserved and not replaced by subsequent calls to show(). Afterwards, the show() function from the Bokeh library is called to display the visualization in a browser tab.

Thus, FR4 is fulfilled by providing separate, readable, interactive visualizations for each dataset and for both types of comparisons.

4.1.5 FR5: Compile SQLite database

SQLAlchemy is used to implement FR5. The initializer of TableClass calls create_engine() from SQLAlchemy, stores the resulting engine, and establishes a connection to it.

After each dataset has been wrapped in instances of TableFunction or TablePoints, these instances are written to separate tables in the SQLite database using method save_to_sql(). The same procedure is applied to the results of both comparisons. To avoid leaving the SQLAlchemy connection open at the end of the program execution, each connection is closed with the method close_sql_connection().

The method save_to_sql() uses the pandas method to_sql(). This method is called on a DataFrame and writes its data into the database. For this call, the SQLAlchemy engine and the table_name attribute of the TableClass are used. Since the datasets wrapped in DataFrame objects already have

the required structure, no changes are necessary before calling to_sql(). This keeps the implementation of save_to_sql very short.

The method close_sql_connection() calls the method close() on the connection, sets the connection attribute to None, invokes dispose() on the SQLAlchemy engine, and finally sets the engine attribute to None.

In addition, the method read_from_sql() is implemented. There is no explicit requirement to read from a SQL database, but this method was added to improve usability and to test save_to_sql(). It uses the pandas function read_sql_table(), which requires the table name and the SQLAlchemy engine as parameters. The return value of this pandas function is a DataFrame containing all data from the table, and this DataFrame is returned by read_from_sql().

Therefore, functional requirement five is fulfilled by exporting all relevant data to an SQLite database via SQLAlchemy and pandas, while managing the database connections in a controlled way.

4.2 Implementation of non-functional requirements

This section explains the implementation of the non-functional requirements, by discussing the arguments for and against the chosen approach.

4.2.1 NFR1 and NFR2: Object-oriented design and Inheritance

Non-functional requirement NFR1 states that the system design must be object-oriented throughout the implementation. NFR2 specifies that at least one inheritance relationship must be used. Object-oriented programming (OOP) is a programming paradigm in which software is structured around objects that combine data and behavior. To follow this paradigm in the implementation, classes are defined as blueprints for the objects.

An important principle of OOP is encapsulation. Data and the methods that operate on this data are bundled together in classes, so that internal details are hidden behind a clear interface.

A second principle of OOP is inheritance. A class can inherit from another class and thereby reuse its data and methods, while adding new behavior and additional data.

This requires careful planning before implementation, because introducing new classes and inheritance hierarchies midway through development can be complex and error-prone, and such mistakes must then be discovered and corrected during testing.

When planning the implementation, the technical requirements also had to be considered. They specify that the program must use the pandas and Bokeh libraries. Both libraries work with their own objects; pandas with DataFrame objects and Bokeh with figure objects. Therefore, the decision was made to apply encapsulation by wrapping the DataFrame and figure objects in dedicated classes.

In addition, inheritance is used for the class that encapsules the DataFrame. The data stored in the DataFrame differs significantly between the two types of datasets used in the program, which motivated the creation of two subclasses. These subclasses clearly distinguish between the types of data stored in the class instances while reusing most of the behavior from the base class, thus supporting both the principle of encapsulation and the principle of inheritance.

NFR3: Exception handling

The non-functional requirement NFR3 specifies that the system must handle both standard and user-defined exceptions. The program validates all inputs in its methods and raises appropriate standard exceptions with descriptive error messages when invalid data is detected. In addition, critical program steps are executed in try-except blocks to avoid low-level, unhandled exceptions from the libraries used. Because no interactive user recovery is required by the assignment, the program terminates in these situations with clear exceptions instead of continuing in an undefined state until a library method fails.

NFR3 is thus satisfied, as silent incorrect results are prevented and clear, interpretable error messages are produced. Program termination in these error cases is acceptable because continuous operation is not required and no user interaction is specified in the assignment.

5 User Perspective

In this chapter, the requirements regarding installation and setup are described to ensure successful execution of the program. Furthermore, the program execution and the usage of the visualizations are briefly explained.

5.1 Installation and setup

The project was implemented and tested with Python 3.14.0. It is therefore required to install Python 3.14 in an editor or development environment of your choosing to run the program as described. Other Python 3.x versions are not officially supported in this project, as they can produce unintended behavior that has not been tested.

In addition, the program depends on the following third-party libraries: (1) pandas, (2) SQLAlchemy, (3) Bokeh, and (4) numpy. These must be installed in order to execute the program. The required packages can be installed individually by using the “pip install” command, for example:

```
pip install pandas sqlalchemy bokeh numpy
```

In addition, the program also uses the standard-library modules pathlib and math for file-system path handling and calculation. These libraries are included with Python and do not require separate installation.

This project is provided as a Python source code repository and is intended to be run in a standard Python development environment. A link to the repository is included in the appendix. The source code follows a src/tests layout: the application code resides in the src package and the automated tests in the test directory.

To set up the project, the following steps are required:

1. Download the project repository.
2. Check out the corresponding branch that contains this project.
3. Open the project root folder named “dataset_function_comperator” in your Python IDE.
4. Ensure that the working directory of your IDE or terminal is set to this project root.

Afterwards, the program can be executed as described in the following section.

5.2 Usage guide

To execute the program from the command line, run:

```
python -m src.main
```

This command treats src as a Python package and starts the application via the main module. No further user interaction is required; the program runs to completion automatically.

At the end of the execution, two messages are printed in the terminal: the first message confirms that all visualizations have been created, and the second message confirms that all SQL connections have been closed and the program has finished execution.

During program execution, six visualizations are displayed in the default web browser. One browser tab is opened for each visualization. The user can interact with all visualizations using the standard tools provided by Bokeh, such as zooming in and out with the mouse wheel and panning by dragging with the left mouse button. For an in-depth description of the available interaction options when working with Bokeh visualizations, the official Bokeh documentation can be consulted.

The first three visualizations show the datasets for ideal functions, training functions, and test points, so that the user can obtain an overview of the given data. These visualizations use a large scale, because some y-values of functions and points range from -16000 to 15000. Most of the data is not of such large magnitude, and zooming is therefore necessary to extract meaningful information from the majority of the data points.

The fourth visualization displays the mapping of the training functions to the ideal functions that are closest to them. This plot contains only three functions, because the scale of the fourth function does not allow for a meaningful visualization. When zooming into this plot, it can be observed how the training functions approximately follow the ideal functions.

The fifth and sixth visualizations show the mapping of the test points to the four ideal functions identified using the training functions. The mapping is split into two visualizations so that the scale of the function in the sixth visualization does not make the mapping for the other three functions unreadable. When zoomed out, the mapped points appear to lie exactly on one of the functions. Only when zoomed in can it be observed that the points are not plotted exactly on the function graphs but lie sufficiently close enough to satisfy the mapping requirements.

6 Testing and Validation

Chapter six first describes the test strategy for the program. Then the test cases get linked explicitly to the functional requirements they verify. Lastly, the known limitations and bugs are discussed.

6.1 Test strategy

A test suite using the unittest framework is implemented in the file test_tables.py to verify the methods directly against their specified requirements. The suite is organized by class: for each TableClass and its subclasses there is a dedicated test class. For every method, the tests cover invalid argument cases such as wrong types, invalid values, and where necessary, out-of-range values, expecting the appropriate exceptions to be raised. In addition, at least one happy path test is provided per method to verify the correct behavior with valid inputs. In total, 36 Tests have been implemented. The test method names always start with the key word “test”, followed by the name of the tested method and either the suffix “happy_path” for tests validating correct behavior or short descriptors indicating which invalid values are being exercised.

Three CSV-files were created as dedicated test data to efficiently test the required comparisons. The files unit_test_functions_1.csv and unit_test_functions_2.csv each contain the data of four functions; all functions except the first one are identical in both files. These functions are shorter than the ones from the assignment but still cover the interval from -100 to 100, providing a sufficient range without becoming too large to inspect easily. The third file unit_test_points.csv, contains five points of which not all are close enough to any function, so that both successful and failing point-to-functions assignments can be tested.

The tests are executed from the project root using Python’s built-in unittest test discovery. With the src/test layout, all tests can be run via:

python -m unittest discover -s test -p “test*.py”

This command discovers the test module test_tables.py in the test directory and runs all test classes and methods whose names follow the Test*/test_* conventions.

6.2 Test cases linked to requirements

FR1: Data import from CSV

The data import is implemented in the class constructor for TableClass, which is reused by the subclasses. The TestTableClass has eleven tests for this constructor testing the correct behavior when creating class objects with a csv file or a dictionary as data source. These include:

- A successful import from a valid CSV file and a successful import from a valid dictionary.
- Failure cases for an invalid file name, invalid parameter types, missing parameters, incorrect column structure, and invalid column content, each of which is expected to raise an error.

FR2: Comparison of mathematical functions

The comparison is implemented in the method find_function_closest() in class TableFunction. There are twelve tests for this requirement, equally spread between the method find_function_closest() and the method compare_functions() a helper function used in find_function_closest().

The tests for find_function_closest() include:

- A successful comparison between two tables.
- Failure cases for invalid parameter types and parameters out of bounds.
- A failure case for two functions with unequal length.
- A failure case for two functions with different x-values.

The tests for compare_functions() include:

- A successful comparison between two tables.
- Failure cases for invalid parameter types and invalid parameter contents.
- A failure case for two functions with unequal length.

FR3: Comparison of mathematical points

Comparing mathematical points with mathematical functions is done in method compare_points(). The method find_highest_deviation() is used beforehand, as the results are needed for the point comparison. Method find_highest_deviation() is tested six times in total for:

- A successful computing of the highest deviation.
- Failure cases using invalid parameter types and out of range parameter values.
- A failure case using a parameter with invalid content length.

The method compare_points() has five tests implemented for:

- A successful mapping to a table of functions.
- An invalid parameter type
- Failure cases using parameter contents without calculated deviations.

FR4: Data visualization

The visualization of all data was not tested using unittests. As this requirement is aimed at user interaction, it was tested by hand.

FR5: Compile SQLite database

A SQLite database is connected by the constructor of TableClass and then written by the method save_to_sql(). There is one test verifying that the data can successfully be saved to SQL using this method, by first saving the data of a TableClass and then loading it with the function read_from_sql(). Another test verifies that the method read_from_sql() only works after the data was first saved to SQL at least once.

6.3 Known limitations and bugs

As user interaction is not required for this program, no user interface is implemented. Therefore, the filenames and paths of the datasets are fixed in the program code. To test the program with other CSV files, the code must be modified manually, or new test cases must be implemented.

Another known limitation is the validation of input data from dictionaries and CSV files. In principle, it would be possible to validate the step size of the x-values and the correct representation of a mathematical function, but this would require a significant implementation effort. Since the program is only required to work with the three given datasets, all of which have the correct structure, this limitation is acceptable and was not addressed. However, when the program is tested with other CSV files, this limitation must be taken into account.

The meaningful visualization of functions is limited, as the plot is always scaled to the function with the largest range of values. Therefore, functions with significantly different scales cannot be visualized together in a meaningful way. When using or testing the program with new data, this can result in visualizations from which no meaningful information can be obtained.

The limited robustness of the database layer must also be considered during testing. The connection to the database is always local and intended for single-user access only.

7 Conclusion and Future Work

Chapter 7 concludes the documentation by summarizing what requirements the program has fulfilled and what was achieved. Further, the lessons learned during implementation, discovered limitations which could be improved, and possible use cases for the program are discussed.

7.1 Summary of what was achieved

The implemented program automatically evaluates and visualizes the relationship between measured data points and a set of ideal functions. It reads the datasets provided as CSV files, validates the data, and performs comprehensive error handling. A comparison algorithm selects, for each training function, the most similar ideal function and computes the corresponding maximum deviation, which is later used to classify test points. Each test point is either assigned to the best matching of the previously found four ideal functions or flagged as not assignable if it exceeds the allowed deviation of all functions.

All tables and results are stored in an SQLite database, and an additional method allows these data to be reloaded for further analysis.

Bokeh-based HTML visualizations are generated that show the ideal, training and test data. The computed comparisons and deviations are also visualized and highlighted to support user inspection.

The implementation follows an object-oriented design: the classes TableClass, TableFunction, and TablePoints encapsulate data loading, validation, comparison logic, and persistence. This design makes the code reusable for different datasets and easier to extend with additional data types or behavior.

7.2 Lessons learned

During testing of the initial implementation for visualizing the functions, the large difference in scale between some functions was noticed. At first this did not appear problematic, but once the function comparison logic was added, it became clear that the scale difference affected both readability and interpretation. A quick fix was implemented by excluding the large-scale function from the visualization. However, after implementing the comparison of points to functions, the resulting plot again became hard to read, and a more systemic solution was required. Ultimately, the visualization methods were extended with an optional set parameter that controls which data are included, so that both function and point visualizations can be configured and reused efficiently.

A second issue with the Bokeh visualization was discovered later during testing. All plots were written to the same HTML file and opened from the same file path in the browser. When the program executed quickly, some plots were overwritten by subsequent ones, so earlier results could no longer be inspected. Two alternative solutions were considered: first, using Bokeh’s tab functionality to display all plots in a single HMTL file; second, generating a separate HTML file each visualization.

The first approach would have provided better user experience with only one browser tab and minimal storage usage but would also have required significantly more implementation effort. In contrast, creating one HTML file per visualization could be implemented quickly with minor change to the existing class and method structure. Given the limited added value of tabs compared with the additional complexity, second solution was chosen and implemented.

7.3 Possible extensions or improvements

The program could be improved by adding mor extensive data input validation. In particular, supporting function data in CSV files with different structures would increase the flexibility and robustness of the application.

An interactive user interface could be added to improve the user experience and allow users to import arbitrary CSV files in a more flexible way. Such an interface would strongly benefit from the extended data validation mechanism mentioned above.

The visualization could be enhanced further by automatically rescaling the axes, adding legends and integrating tool tips. These features would enhance the user's experience when using new datasets.

7.4 Possible use cases

There are two broad areas of application for a data analysis program like this one: modeling the behavior of physical systems and verifying observations against expected patterns.

Examples where real-world data can be compared to mathematical functions:

- Engineering experiments that measure data (e.g., radioactive decay, cooling curves) and fit them to theoretical functions such as exponentials, sines, or polynomials to estimate parameters and verify the underlying physical model.
- Signal processing scenarios where sensor data from audio, or vibration measurements are modeled with periodic functions to analyze frequency content and detect periodicity.
- Economic applications in which historical data such as sales, stock prices, or demand are fitted with regression functions to understand trends and customer behavior.
- Biological and chemical experiments where reaction rates or growth curves are fitted to nonlinear functions to estimate parameters and study how a system responds to inputs.

Typical situations in which individual data points are compared to mathematical functions include:

- Quality control in manufacturing, where each product's measurements are compared with a nominal tolerance curve. Measurements within the allowed deviation are accepted; those outside are rejected.
- Safety limit checking can be done by comparing the measured data points against a regulatory safety limit curve. If a point crosses the curve, it is marked as violating the limit.

Appendix

The appendix contains a link to the source code repository from which the program “dataset_function_comperator” can be downloaded and used as described in chapter 5.

The repository contains this documentation, a text-file containing the code for the additional Git task, and the program code with the project root being the folder “dataset_function_comperator”.

The repository is publicly available here: https://github.com/Lebix02/Programming-with-Python